G L BAJAJ COLLEGE OF TECHNOLOGY AND MANAGEMENT, GREATER NOIDA

A Project Report
On

UniCare Hospital Management System

Submitted in partial fulfillment of the requirement for the award of the degree of Master of Computer Applications

By

Sindhraj Singh
Roll No. – 2412000140166

Under the Supervision of

Ms. Vandana Kumari
(Assistant Professor)

DR. A P J ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW
(2025-26)

PROJECT REPORT ON

UniCare Hospital Management System

(BMC-452)
Session-2025-2026

Department of Master of Computer Applications (MCA)

Submitted to:
Ms. Vandana Kumari
(Assistant Professor)

Submitted by:
Name: Sindhraj Singh
Roll no: 2412000140166
Semester: 4th
Section: B3

G L Bajaj College of Technology & Management
Plot No 2, APJ Abdul Kalam Rd, Knowledge Park III,
Greater Noida, Uttar Pradesh

ACKNOWLEDGEMENT

I would like to thank everyone who helped me in completing my major project titled “UniCare Hospital Management System”.

First of all, I would like to thank my project guide, Ms. Vandana Kumari (Assistant Professor), for her invaluable guidance, constant support, and encouragement throughout the development of this project. Her expertise and timely suggestions were instrumental in successfully completing this work.

I would also like to extend my sincere gratitude to the faculty members of the Department of Master of Computer Applications at GL Bajaj Institute of Technology and Management, Greater Noida, for providing the necessary resources, lab facilities, and academic support.

I am deeply thankful to my classmates and peers who contributed their ideas, feedback, and motivation during the project development phase.

Finally, I express my heartfelt gratitude to my family for their unwavering support, patience, and encouragement throughout this academic journey.

(Signature)

Sindhraj Singh
2412000140166

CERTIFICATE OF ORIGINALITY

I hereby declare that my Project titled "UniCare Hospital Management System" submitted to **Dr. APJ ABDUL KALAM TECHNICAL UNIVERSITY, Lucknow** for the partial fulfillment of the degree of Master of Computer Applications Session 2025-2026 from **G L Bajaj College of Technology and Management, Greater Noida** has not previously formed the basis for the award of any other degree, diploma or other title.

Place: Greater Noida **Signature**

Date: 11/04/2026 Sindhraj Singh

CERTIFICATE OF ACCEPTANCE

This is to certify that the project entitled, "**UniCare Hospital Management System**" submitted by **Sindhraj Singh** a bonafide student of **G L Bajaj College of Technology and Management, Greater Noida** in partial fulfillment for the award of **Master of Computer** Applications affiliated to Dr. APJ ABDUL KALAM TECHNICAL UNIVERSITY, **LUCKNOW** during the year 2025-26. It is certified that all corrections, suggestions indicated as per Internal Assessment have been incorporate in the project.

To the best of our knowledge, the work embodied in this report is original and has not been submitted to any other degree of discipline. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said degree.

Mrs. Vandana Kumari
(Assistant Professor)

[Sign of External Examiner]

Head of Department
Master of Computer Applications
G L Bajaj College of Technology and Management, Greater Noida

TABLE OF CONTENTS

CERTIFICATE OF ORIGINALITY 2
CERTIFICATE OF ACCEPTANCE 3
TABLE OF CONTENTS 4
LIST OF FIGURES 5
LIST OF TABLES 6
Chapter 1 7
Chapter 2 10
Background Study and Research Gap 10
Chapter 3 12
Chapter 4 17
Proposed work and Methodology 17
Chapter 5 20
Design/Development 20
Chapter 6 27
Testing and Implementation 27
Chapter 7 30
Result and Discussion 30
Chapter 8 33
Conclusion and Future Scope 33
REFERENCES 36

Anexure-1
Plagiarism Report and AI-generated Content Report 37
Anexure-2
Additional Data/ Resources if used 38

LIST OF FIGURES

Fig. 1.1 - Proposed System Architecture of UniCare HMS

Fig. 1.2 - Use Case Diagram of the System

Fig. 1.3 - Level 0 Data Flow Diagram (Context Level)

Fig. 2.1 - Level 1 Data Flow Diagram

Fig. 2.2 - Entity Relationship (ER) Diagram

Fig. 2.3 - Database Schema and Table Relationships

Fig. 3.1 - Role-Based Login Interface (Admin, Doctor & Patient)

Fig. 3.2 - Admin Panel - Patient & Doctor Management Dashboard

Fig. 3.3 - Patient Portal - Appointment & Symptom Checker View

Fig. 4.1 - System Hardware and Software Requirements

Fig. 4.2 - Automated Email Notification via Nodemailer (SMTP)

Fig. 4.3 - Google Gemini AI Symptom Checker Integration

LIST OF TABLES

Table 1.1: Admin Table Schema

Table 1.2: Patient Table Schema

Table 1.3: Doctor Table Schema

Table 1.4: Appointments Table Schema

Fig. 2.1 - MySQL Workbench View of Appointments Table

Fig. 2.2 - MySQL Workbench View of Medical Records Table

Chapter 1

Introduction and Aim of the Project

1.1 Introduction

Today, health organizations are incorporating automated and intelligent software to replace manual processes in handling, recording, and administering patients. The efficient management of the resources within the hospital plays a crucial role within the healthcare system. This is because the delivery of prompt and accurate information concerning appointments, availability of physicians, prescriptions, and billings among others, to patients and other hospital personnel is extremely vital.

UniCare Hospital Management System, henceforth called UniCare HMS, is a smart, cloud-enabled, web-based solution intended to help digitize and automate all processes involved in a hospital. In the past, hospitals were operating using manual processes for appointment booking, keeping patient details manually, and having separate databases for different departments. These practices often resulted in data redundancy, bureaucracy, medical mistakes, and even delay in providing healthcare services.

For its backend, the system employs the Node.js and Express.js framework. This makes the system scalable and responsive in real time. The Symptom Checker application is powered by Google Gemini AI, and it uses patient symptom reports to predict possible diseases, alongside suggesting the best department to visit at the hospital. Data is stored on the MySQL relational database management system. Email notifications are sent automatically using the Nodemailer module.

1.2 Motivation of the Project

The main reason behind the conception of this project UniCare HMS is inefficiencies caused by using manual systems for hospital management. In today's hectic healthcare industry, there is loss of time on behalf of both patients as well as employees because of paperwork, scheduling, and disjointed record keeping. Furthermore, UniCare HMS was also prompted by the revolutionary impact of artificial intelligence on the healthcare sector. Using the AI-powered symptom analyzer, the system not only converts paper-based files into electronic format but also offers clinical intelligence through it.

1.3 Problem Statement

Management of hospitals in the traditional way faces many challenges, such as the following:

- **Data Inconsistency:** Medical files, medications, and other information of patients are saved separately from each other.

- **Appointment Scheduling Through Physical Visits:** Patients are required to physically visit the hospital for appointment bookings.
- **Real-Time Accessibility Issues:** Employees can't immediately verify the availability of doctors, beds, or rooms.
- **Missing Automated Notifications:** Patients do not get automated reminders regarding their upcoming appointment schedules, causing frequent no-shows.
- **Poor Analytical Tools:** Hospital management does not have any dashboard tools for assessing the inflow of patients, income trends, or doctor productivity.

1.4 Project Purpose

Overall, UniCare HMS seeks to develop an integrated platform for hospital management that will ensure precise and immediate dissemination of data to all parties involved – patients, doctors, nurses, and administrative staff – via a secure web portal and automatic e-mail notification.

1.5 Objectives

The project consists of the following goals:

- **Multi-Role Secure System:** Design an application that will include roles for Administrators, Doctors, Nurses, and Patients with proper authentication and access privileges.
- **Symptom-Based Analysis with AI:** Incorporate the Google Gemini API to provide dynamic analysis of symptoms provided by patients and suggest suitable departments.
- **Notification System:** Utilize Nodemailer to automate notification processes such as appointment confirmation, reminder, and billing receipts.
- **Data Storage and Management:** Create a normalized MySQL database for data storage such as patient, doctor profile, appointment, prescription, and billing details.
- **User Interface:** Design a business professional user interface responsive for desktop, tablet, and mobile devices.

1.6 Project Scope

The UniCare HMS has the following scopes:

- **Tech scope:** The platform is built using MEEN Stack (MySQL, Express.js, Gemini AI, Node.js) and RESTful API design architecture.
- **Function Scope – Admin:** Full CRUD functionalities for patients, doctors, departments, bed management, billing, and analytics dashboard.

- **Function Scope – Doctor:** Access to assigned appointments, updating of patient records, and issuing electronic prescriptions.
- **Functional Scope - Patient:** Schedule appointments, review patient records, AI-based symptom checker, and retrieve invoices.
- **Security Scope:** Session management, password hashing using bcrypt, and sanitization to protect against SQL injection and cross-site scripting (XSS).

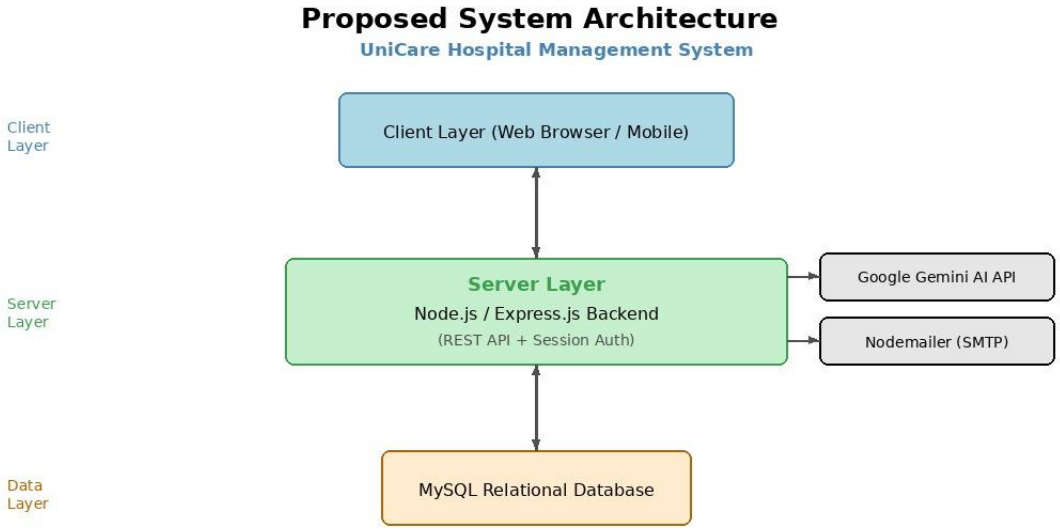

Fig. 1.1 - Proposed System Architecture

1.7 Technologies Used

In order to comply with industry norms, the below technologies are utilized:

- Backend: Node.js and Express.js (Asynchronous I/O, High concurrency).
- **Database:** MySQL (Relational Database, ACID properties).
- **AI Model:** Google Gemini Pro API (NLP, symptom summarization).
- **Communication:** Nodemailer (Email alerts, SMTP).
- **Frontend:** HTML5, CSS3, JavaScript.

Chapter 2

Background Study and Research Gap

2.1 Background to this Study

In recent years, there has been a major revolution within hospital and healthcare administration systems. From the perspective of technology, the change has been from manual registers to digital registers. Despite this, the core aim has been consistent in that it is all about providing the appropriate medical data to the correct stakeholders at the right time. This, however, has not been the case for most hospitals, particularly those operating in tier-2 cities and hospitals affiliated to schools.

2.2 History of Higher Education Communication

Historically, the development of hospital management systems is marked by different eras, including:

- **The Manual Era:** Where patient information would be kept in physical registers, booking logs, and handwritten prescription slips. Loss of data, illegible handwriting, and data duplication were very frequent problems.
- **The Early Computerized Era:** Desktop applications that were used to handle billing processes or manage pharmacology, operating independently of other departments within the hospital.
- **The Internet Age:** The advent of websites for patient booking was introduced, although limited to scheduling, and not integrated with patient records and AI technologies.

2.3 Research on the Technology: Stack Choice

For order to implement a modern solution, extensive research was conducted regarding the technological state. The choice of using MEEN Stack architecture (MySQL, Express, Node.js) and AI integration was made on the grounds of the following research.

2.3.1 Research into Node.js and Express

According to the research conducted on back-end technology, it has been revealed that Node.js, using Google's V8 engine for executing the JavaScript language, is wholly compatible for developing real-time applications. Being a non-blocking and event-driven system, Node.js can handle multiple requests from doctors and patients simultaneously without any delays. This becomes very essential especially during peak hours like morning OPD rush.

2.3.2 Relational Database (MySQL)

Even though the trend seems to point towards the usage of NoSQL databases, there has been a lot of emphasis placed on MySQL from an academic perspective when it comes to healthcare information structures. Information pertaining to medical files, patient records, schedules of doctors, billing, etc. is well-structured and connected. MySQL ensures that ACID properties allow for immediate consistency.

2.3.3 Large Language Models (LLMs)

One of the key themes explored through literature review is the use of artificial intelligence in health care triage. Studies have shown that patients usually have difficulties in determining the specialist they should seek for their particular symptoms. The Google Gemini Pro API is able to understand human language well enough to be able to interpret the symptoms expressed in English and find possible medical illnesses and the departments that should be consulted.

2.4 Research Gap

Even with the presence of some digital technologies, the following critical gaps still exist in hospital management:

The following gaps have been identified:

- **AI Triage Gaps:** The current software provides appointment booking forms but lacks an intelligent system that can advise patients on how to select the correct department or doctor.
- **Push Notification Gaps:** Current hospital software relies on the pull method; patients need to login to check any notifications. There is a need for push notification software where appointment booking and payment reminders are sent to the patients' email addresses.
- **Analytics Gaps:** Hospital administrators using current software lack a dashboard that would give insights into patient flows, doctor efficiency, and department load.
- **Integration Gaps:** Scheduling, billing, medical record keeping, and pharmacy management are independent systems.

2.5 Solution to Bridge the Gap

The UniCare Hospital Management System caters to these weaknesses by leveraging the strength of MySQL, powerful Node.js, advanced Google Gemini AI, and active communication by Nodemailer. The UniCare HMS helps create a smart hospital ecosystem through the combination of these technologies, transforming the ordinary hospital portal into a cognitive assistant, which takes care of everything from triage to notifications, all on one platform.

Chapter 3

Tools/Platform, Hardware and Software Requirement Specification (DFD, ER/UML)

3.1 Introduction

System Design and Requirement Specification is the design process behind the UniCare Hospital Management System. In this chapter, the design of the platform that will be used to develop the system will be specified. Tools and technologies that will be used to implement the system will also be specified. The data model will be specified using Data Flow Diagram (DFD) and Entity Relationship (ER) Diagrams.

3.2 Tools and Platforms

These technical tools were selected due to their performance and flexibility in terms of integrating with AI-based services and relational data model.

3.2.1 Development Environment

Operating System: Windows 10/11 or Linux (Ubuntu).

Text Editor: Visual Studio Code (VS Code) was used because of its strong capabilities in terms of JavaScript coding, built-in terminal, and debugger extensions for Node.js.

Version Control: Git/GitHub is used to manage modifications to the source code and for version control.

3.2.2 Backend (Node.js & Express.js)

The Node.js programming language provides a non-blocking input/output operation that is suitable for a notification application in a hospital where multiple users can access the application at any one time. The Express.js framework is used to route requests and provide middleware for authentication and authorization purposes.

3.2.3 Database (MySQL)

MySQL is used for storing structured information related to the hospital, such as patient information, doctor details, appointment schedules, medical information, prescription details, and billing records. It provides a robust system for relational mapping that ensures that complex SQL operations, like "Give me all patients of Dr. Mehta for this week," are handled reliably and quickly.

3.2.4 APIs and Libraries

API for Google Gemini AI: Used for natural language processing (NLP) to process patient symptoms and direct patients to respective departments.

Nodemailer: A Node.js module for sending emails using SMTP protocols.

Multer: A middleware module to manage file uploads, including lab results and prescriptions in PDF or image formats.

Bcryptjs: Password hashing and salting.

3.3. Hardware and software requirements

Minimum requirements needed for the system to operate properly in the real world (college server) are the following:

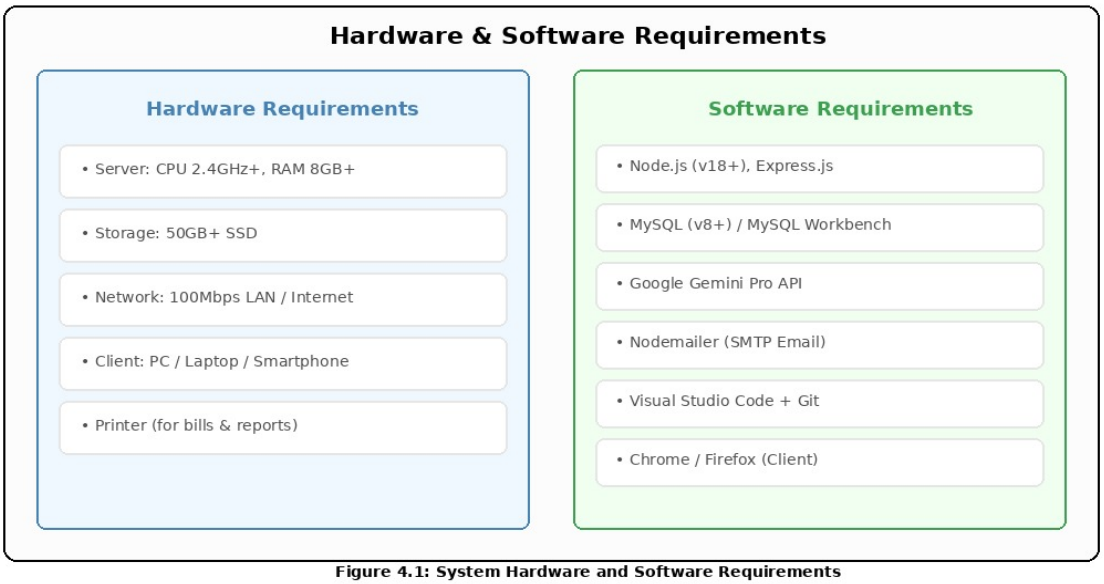

Fig. 4.1 - System Hardware and Software Requirements

3.4 Data Flow Diagrams (DFD)

The DFD shows the process flow within the system, which starts from the admin and doctor inputs, goes through patient communication, AI processing, and finally automatic alerts.

3.4.1 Level 0: Context Level DFD

This is the level where the system is viewed as one complete process.
Admin supplies the patient-doctor information and configuration.
Doctor submits the prescription and medical records.
Patient supplies login details and appointment/symptom request.
System sends back the appointment, symptom analysis by AI, bill, and email notification.

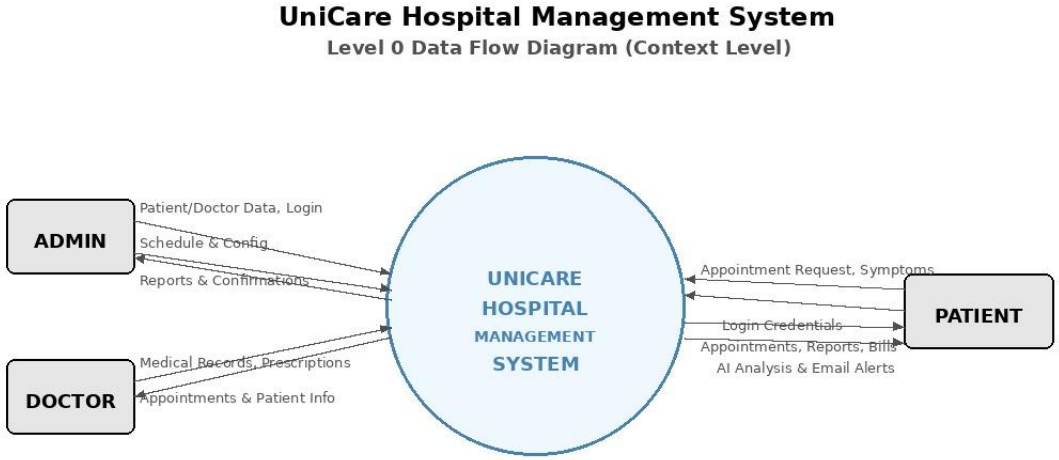

Fig. 1.3 - Level 0 Data Flow Diagram (Context Level)

3.4.2 Level 1: Process Level DFD

It is made up of six major processes, namely:

Authentication (1.0) – Authenticates all users;

Appointment Management (2.0) – CRUD for appointments;

Medical Records (3.0) – Deals with diagnosis and prescription;

Billing (4.0) – Invoice generation and monitoring;

AI Symptom Checker (5.0) – Transmits symptoms to Gemini and gets its analysis;

Email Notification (6.0) – Activates Nodemailer.

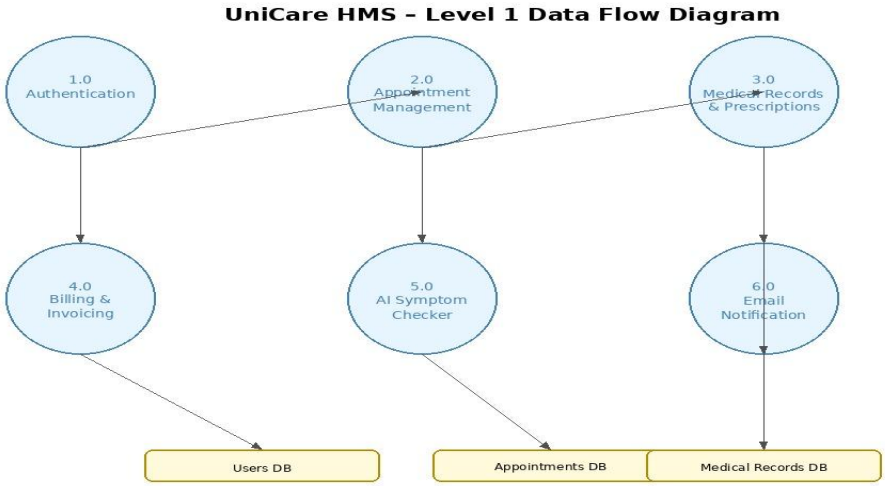

Fig. 2.1 - Level 1 Data Flow Diagram

3.5 Entity Relationship (ER) Diagram

The Entity Relation Diagram denotes the database design. Key entities include:

ADMIN (System management and doctors),

PATIENT (Appointment scheduling and record viewing),

DOCTOR (Appointments and medical record documentation),

APPOINTMENT (Connection between the patient and doctor),

MEDICAL RECORD (Diagnosis and prescription),

BILLING (Per-patient invoice).

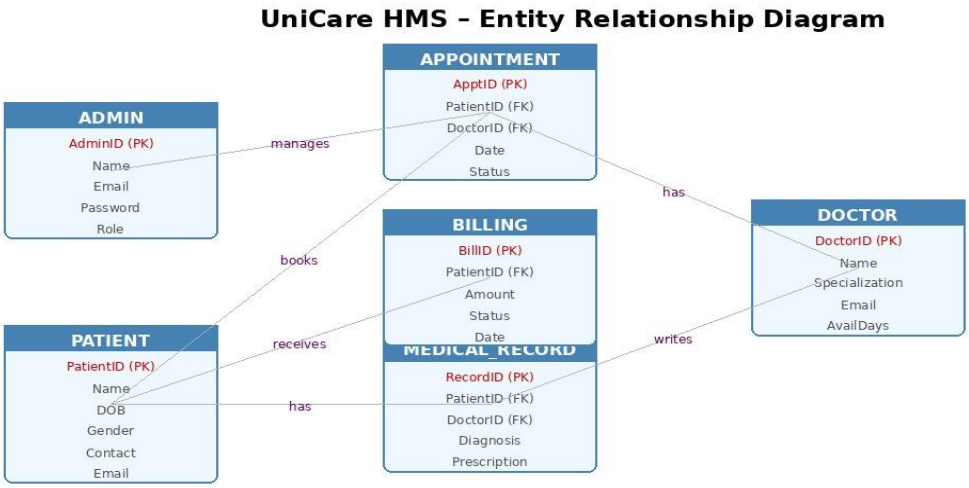

Fig. 2.2 - Entity Relationship (ER) Diagram

3.6 UML Use Case Diagram

Use Case Diagram – This Use Case is all about actors and their objectives.

Admin Actor – Patient management, Doctor management, Billing, Analytics

Doctor Actor – Appointment viewing, Medical record management, Prescribing Medication

Patient Actor – Appointment booking, AI Symptom Checker, Bill Download, Email Alerting

Gemini AI Actor – External actor that does Symptom Analysis.

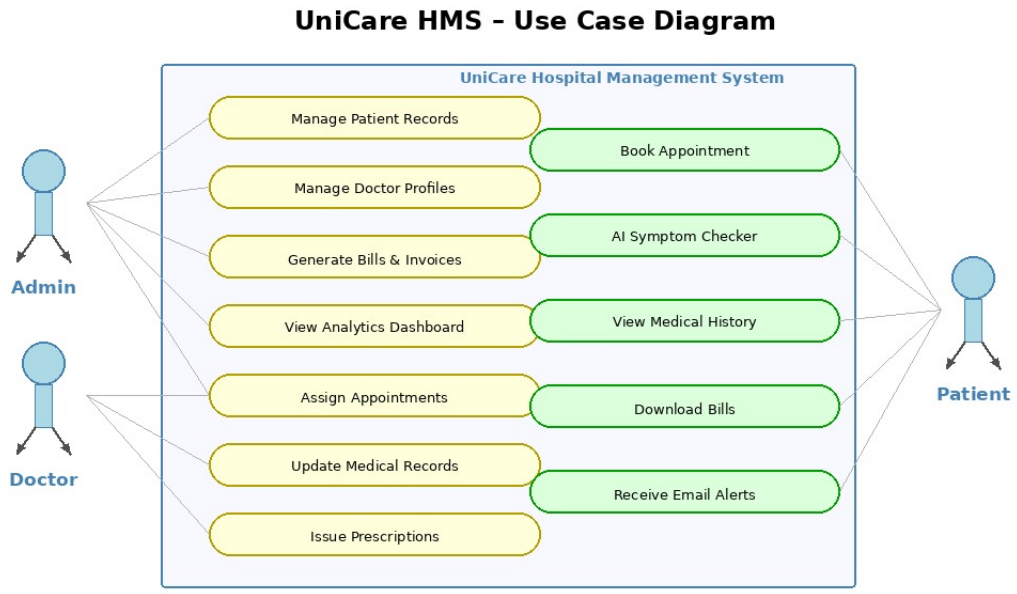

Fig. 3.6 - Use Case Diagram of the System

Chapter 4
Proposed work and Methodology

4.1 Introduction

The objective of this project is to create a smart, automated and AI-enabled hospital management system. UniCare HMS stands out in that it is not a typical hospital portal, but rather one which is predictive and analytic. This process of creation ensures that the project develops logically from conceptualization all the way to implementation.

4.2 Software Development Methodology (Iterative Model)

This model was applied in the current work to facilitate the development process through small steps:

- **Planning and Requirement Analysis:** The main features such as patient registration, appointment booking, and physician login were determined initially.
- **Core Module Development:** The basic functionalities involving creation, reading, updating, and deleting patient and appointment information were implemented using Node.js and MySQL technologies.
- **Integration of AI and Notifications:** Google Gemini AI (symptom evaluation) and Nodemailer notifications (email alerts) were incorporated into the stable core module.
- **Testing and Debugging:** Each step was tested for errors in API calls, database queries, and sessions.

4.3 Proposed System Architecture Workflow

The design of this system follows a layered approach involving frontend, backend, and AI services. The steps involved are:

Step 1: User Authentication: Admin, Doctor, or Patient logs into the system based on their roles through the role-based authentication login process. Express-session is used for managing sessions.

Step 2: Patient Symptom Entry (AI Process): Patients enter their symptoms in plain text. The backend creates a structured input request for Google Gemini Pro and receives probable conditions, suggested specialty department, and priority.

Step 3: Booking an Appointment: The patient chooses a doctor from the suggested specialty department and schedules an appointment. The backend checks the doctor's availability from the MySQL database and verifies the appointment scheduling.

Step 4: Email Confirmation: An automated confirmation email is sent by Nodemailer to the patient confirming their appointment details such as the doctor, date, time, and OPD room number.

Step 5 – Doctor's Portal: The doctor signs in to check appointments, patient medical history, diagnose diseases, and create prescriptions online.

Step 6 – Billing and Analytics: The administrator creates invoices for consultations and treatments. The analytical panel provides key performance indicators like patient numbers, income, and booking statuses.

4.4 Modular Description

The enable the code to be easily maintainable and scalable by splitting it into four main modules:

4.4.1 Admin Management Module

This is the control center of the entire system.

It comprises CRUD functionalities for patients, doctors, and departments, file uploading for labs and prescriptions with Multer middleware, and a read analytics dashboard with information regarding appointment details, revenue details, and doctor details.

4.4.2 Student Dashboard Module

End-user module that is personalized.

This includes AI Symptom Checker, appointment booking facility with slot availability, viewing medical history (diagnosis & prescriptions), and downloading bills in PDF format.

4.4.3 AI Intelligence (Gemini Integration)

Doctor-specific perspective. Includes the list of appointments today with patient details, prescription writer, updating patient medical records after consultation, and full patient history from the assigned patient list.

4.4.4 Email Module (Nodemailer)

This is the "brain" of the entire system. Natural Language Processing via Gemini Pro API processes the patient symptoms. Prompt Engineering makes sure that the responses are action-oriented with probable diagnosis, department suggestions, and urgency levels mentioned.

4.5 Technical Implementation Logic

- **Password Hashing:** bcryptjs is utilized for hashing passwords with salts. No plain text passwords are ever stored.
- **Session Handling:** express-session manages authentication sessions with role restrictions.

- **Asynchronous Functions:** All calls to AI APIs and email sending functions are asynchronous to avoid delay experiences for users.
- **Data Consistency:** Foreign key relationships between appointments, patient records, and invoices in MySQL ensure cascading deletes maintain consistency.

4.6 Approach to Improving Performance

- **Cache AI Answers:** Results of symptom analysis are stored in the database after the first request and pulled from there on future visits, which saves on API expenses and speeds up answers.
- **Responsive User Interface:** CSS Flexbox and Media Queries guarantee that the UI is as functional on a 6-inch phone screen as on a 27-inch one.

Chapter 5
Design/Development

5.1 Development Phase

The development stage entails converting design ideas and system specifications into a functional hospital management application. The key issue during development was establishing an uninterrupted flow between the back-end processes of the hospital administrator and the front-end interface of the patients, while implementing AI and automatic notifications simultaneously.

5.2 Architectural Model (MVC Pattern)

It consists of MVC architecture design patterns:

- **Model (MySQL):** Data access component. Tables such as admin, doctor, patient, appointment, medical_records, and billing tables are created using the DDL scripts in SQL. The model performs all Create Read Update Delete activities.
- **View (Frontend):** This represents the user interface. Developed using HTML5, CSS3, and JavaScript. The card view display is used to display hierarchical information on appointments and patient records.
- **Controller (Node.js/Express):** It represents business logic component. On receiving symptoms from a patient via the UI, the controller requests Gemini API and displays the recommendation from the Gemini API in the View.

5.3 User Interface (UI) and User Experience (UX)

User experience is emphasized extensively. Given that patients will interact with the software through their mobile phones, the design process utilizes a Mobile-First technique.

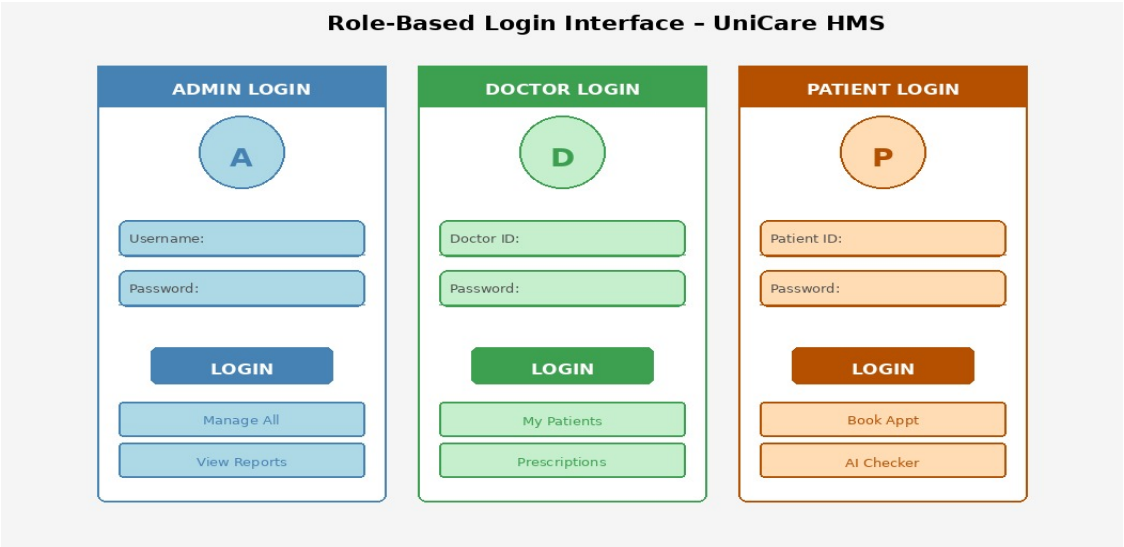

Fig. 3.1 - Role-Based Login Interface (Admin & Student)

5.3.1 Design Principles

- UniCare Hospital Management System shows how advanced today's technology can help with practical problems related to healthcare administration management through automation and solutions. The software proves that the use of AI when appropriately incorporated into hospital system operations helps achieve more within less time and improves the effectiveness and safety of patient management. With the increasing number of patients at the institution, UniCare HMS will still be its core.The home page of all the role portals displays only the essentials; like the appointments of today and any alerting information.

- **Color Coding According to Role:** Admin portal displayed in blue color, the doctor portal in green color, and the patient portal in teal color.

- **Responsiveness:** Thanks to CSS media queries, layouts containing more than one column are converted into single-column layouts while using mobile phones.

5.3.2 Interface Components

Admin Dashboard: Comes with an advanced text editor for composing notifications and provides "Read-Receipt" statistics.

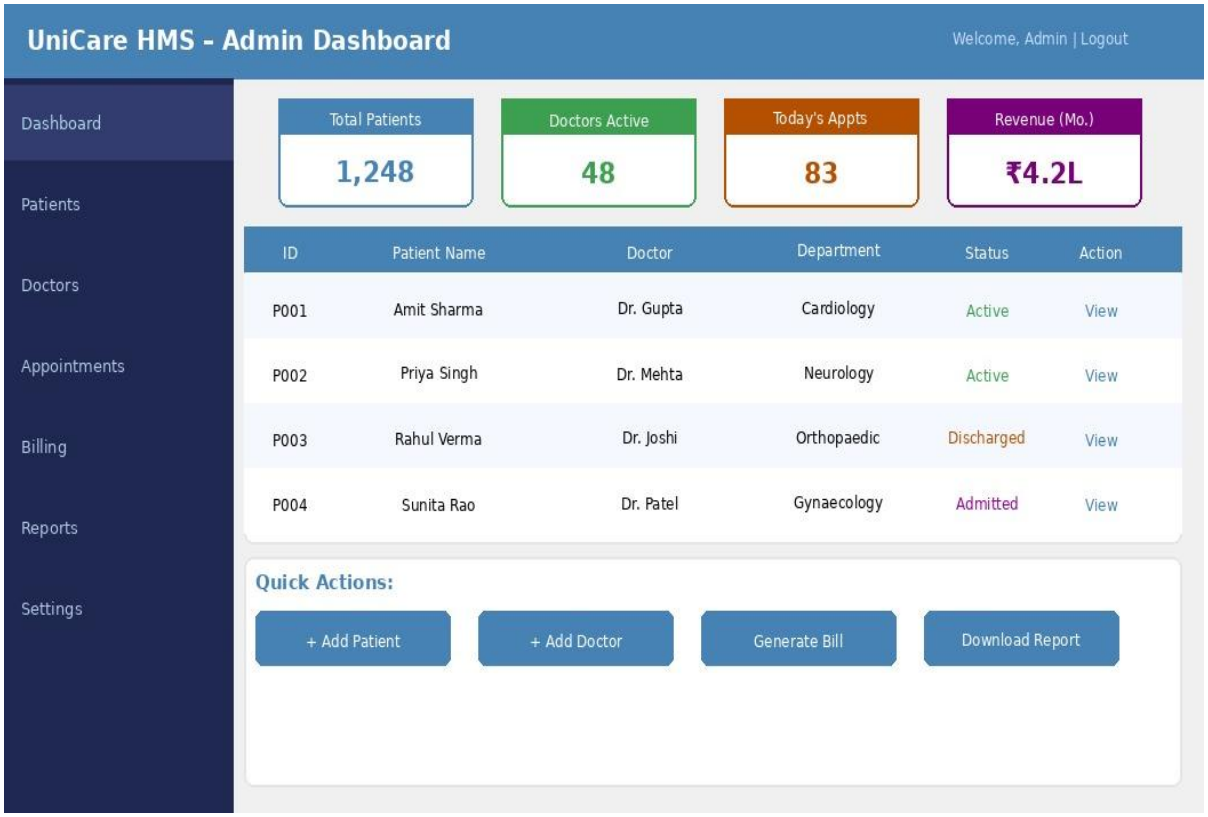

Fig. 3.2 - Admin Panel - Notice Publishing & Management

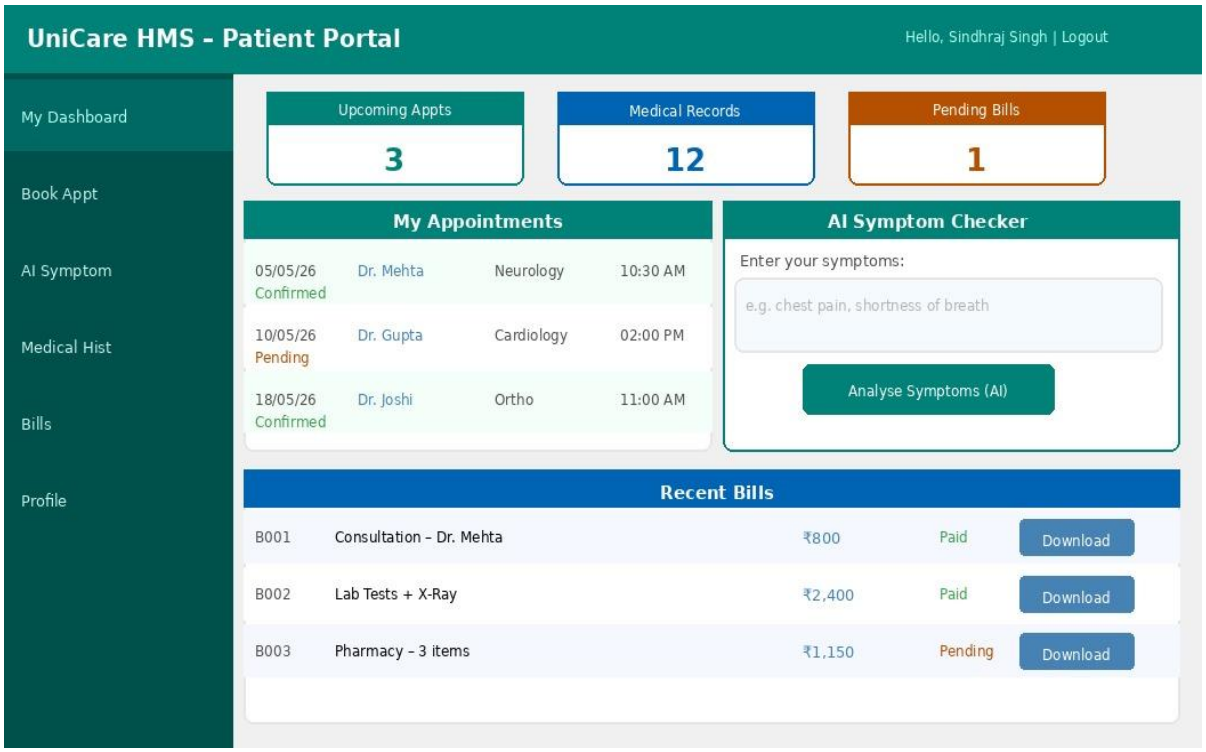

Fig. 3.3 - Patient Portal - Appointment & Symptom Checker View

5.4 Database Design (DDL & DML)

MySQL database is entirely relational in nature. The tables were created through DDL statements generated from the ER model formulated in Chapter 3.

Column Name	Description
admin_id (PK)	Auto-incremented primary key for each admin
name	Full name of the administrator
email	Unique email address used for login
password	bcrypt-hashed password (never stored in plain text)
role	ENUM: ‘superadmin’ or ‘admin’

Table 1.1: Admin Table Schema

Column Name	Description
patient_id (PK)	Auto-incremented primary key for each patient
name	Full name of the patient
dob	Date of birth for age calculation and records
gender	ENUM: ‘Male’, ‘Female’, ‘Other’
contact	10-digit mobile number
email	Unique email address for login and notifications
blood_group	Patient’s blood group (e.g., B+, O-)

Table 1.2: Patient Table Schema

5.4.1 Schema Realization

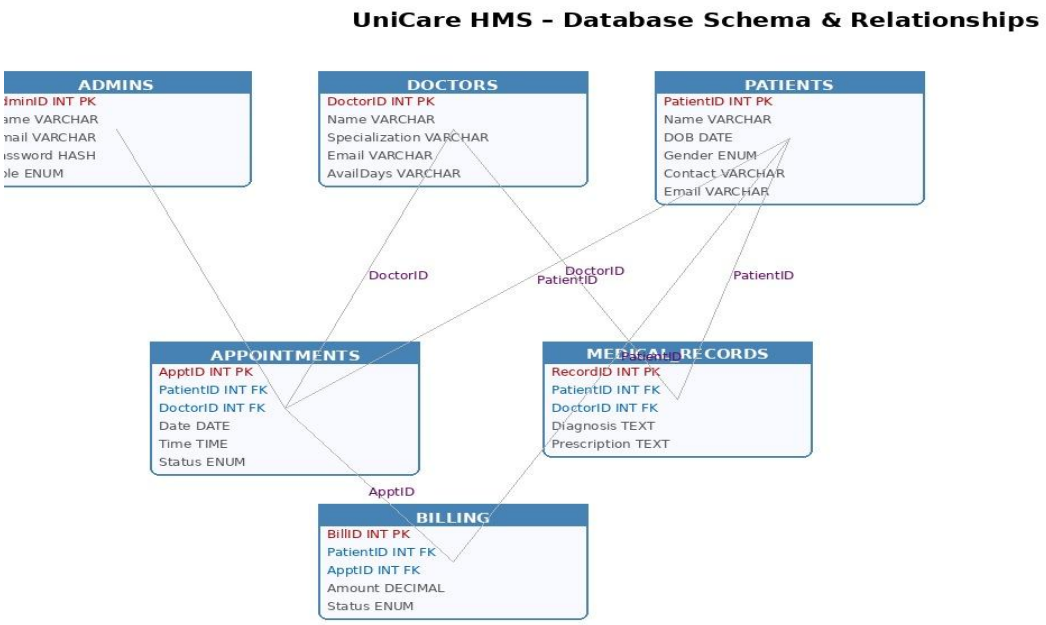

Fig. 2.3 - Database Schema and Table Relationships

MySQL Workbench - unicare_db > appointments

appt_id	patient_id	doctor_id	appt_date	appt_time	status	created_at
1	P001	D003	2026-05-05	10:30:00	Confirmed	2026-04-25 09:12
2	P002	D007	2026-05-10	14:00:00	Pending	2026-04-25 10:45
3	P004	D012	2026-05-18	11:00:00	Confirmed	2026-04-26 11:22
4	P001	D005	2026-05-20	09:15:00	Cancelled	2026-04-27 08:00
5	P003	D009	2026-05-22	15:30:00	Confirmed	2026-04-27 14:30
6	P005	D003	2026-05-25	10:00:00	Pending	2026-04-28 09:00

Fig 2.1: MySQL Workbench View of Appointments Table

Fig. 2.1 - MySQL Workbench View of Appointments Table

MySQL Workbench - unicare_db > medical_records

record_id	patient_id	doctor_id	diagnosis	prescription	date
1	P001	D007	Migraine	Tab Sumatriptan 50mg	2026-04-10
2	P002	D003	Hypertension	Tab Amlodipine 5mg	2026-04-15
3	P004	D012	Vitamin D Deficiency	Cap D3 60K weekly	2026-04-18
4	P001	D005	Lumbar Spondylosis	Physiotherapy + Pain	2026-04-20
5	P003	D009	Type 2 Diabetes	Metformin 500mg BD	2026-04-22
6	P005	D003	Anxiety Disorder	Counselling + SSRI	2026-04-26

Fig 2.2: MySQL Workbench View of Medical Records Table

Fig. 2.2 - MySQL Workbench View of Medical Records Table

5.5 Core Backend Development

Backend: Node.js is used as the backend technology because it employs non-blocking I/O operations to do most of the work behind the scenes, including API requests and emails.

5.5.1 Google Gemini AI – Symptom Checker

API Requests: The system employs the @google/generative-ai Node.js library. An intricately designed prompt like "A patient describes: [symptoms]. Determine likely diagnoses, suggest an appropriate specialty department, and assign urgency as Low/Medium/High." is fed to Gemini Pro.

Tokenization: Output from Gemini Pro is tokenized and stored in structured JSON format in the database, guaranteeing that the same output can be accessed immediately during future visits without making any further API requests.

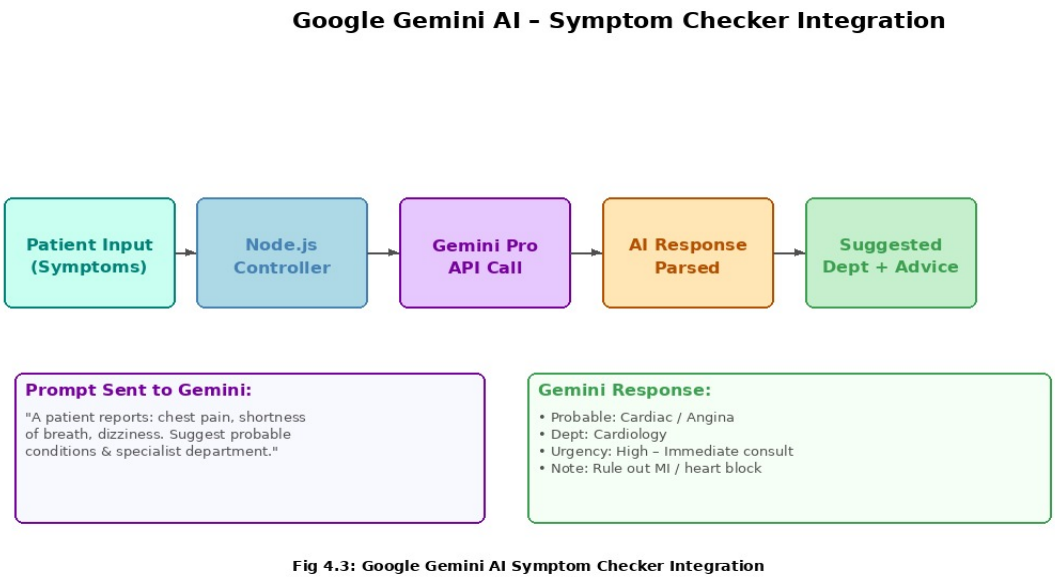

Fig. 4.3 - Google Gemini AI Symptom Checker Integration

5.5.2 Email Engine (Nodemailer)

SMTP Transport: A secure Gmail SMTP relay with App Password authentication is used for high deliverability. Content Generation: The email body includes a personalized salutation, details of the appointment (doctor's name, date, time, OPD room), AI symptoms (if applicable), and a direct link to the patient portal.

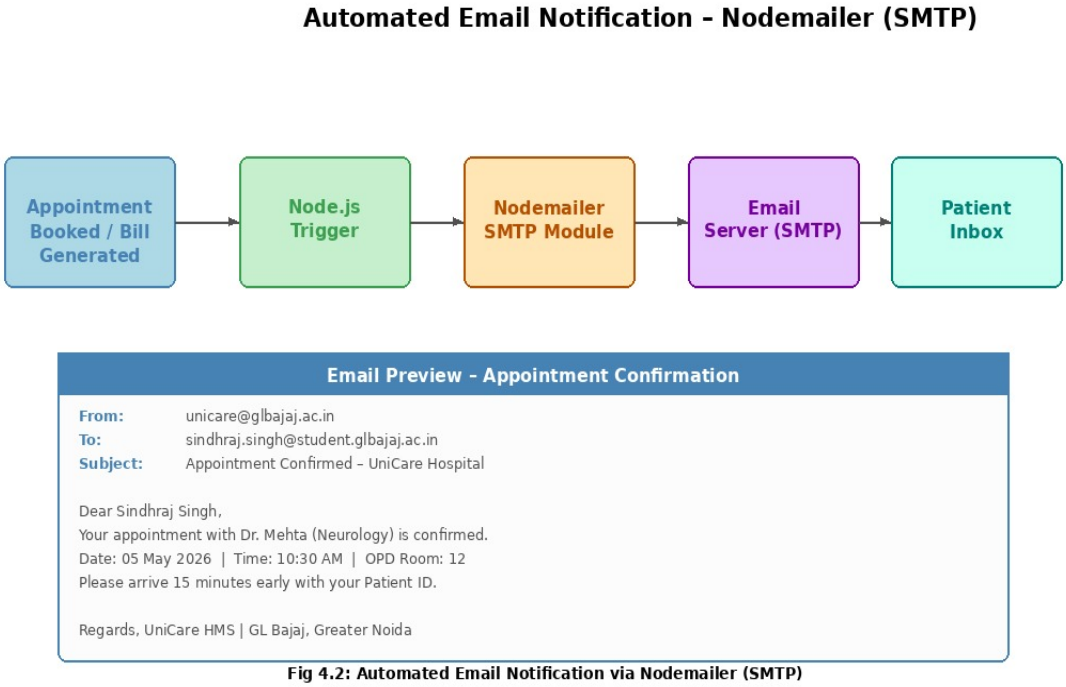

Fig. 4.2 - Automated Email Notification via Nodemailer (SMTP)

5.6 Security and Validation Logic

- **Password Protection:** bcryptjs with 12 rounds of salting makes passwords unbreakable by hashing them.
- **Input Validation:** All input received from users, which includes symptoms inputted by the patient and admin form data, is validated to prevent SQL injection and XSS attacks.
- **Limitations on File Upload:** Multer only allows files in PDF, JPEG, and PNG format up to 10 MB.
- **Route Access Control based on Role:** The Express middleware validates the role from the session before rendering the route for admin, doctors, or patients.

5.7 Error Handling and Logging

The global error handling provided by Express makes the application "crash-proof." In case the Gemini AI API does not respond or takes a long time to respond, the error will be recorded gracefully, and the appointment scheduling process can still go on without symptom analysis.

Chapter 6
Testing and Implementation

6.1 Understanding Software Testing

Testing is a vital phase in the SDLC that ensures the system's security, functionality, and effectiveness. In this case, considering that this is an important system that requires high-quality software, testing takes place at the point where creation meets implementation. This process is not simply about identifying flaws; rather, it involves ensuring that all necessary functions are carried out efficiently. From basic operations like creating, reading, updating, and deleting data to advanced functionalities like analyzing symptoms through artificial intelligence and sending automatic emails, everything should be executed perfectly.

6.2 Testing Strategies and Methodologies

To develop quality software, several testing techniques were adopted. These testing techniques can be employed to perform tests on the system at a high level as well as the code level.

6.2.1 Black Box Testing

In this approach, the code used in the system was considered as a "black box" and there was no interest other than the input and output. Testing user interaction was performed to make sure that the system works as expected. As an example, testing the sign-in process using valid or invalid logins for all three users was done. End-to-end testing of appointment booking was performed from entering symptoms until receiving the booking email.

6.2.2 White Box Testing

The testing was implemented for the backend using Node.js and Express. Focus was placed on path testing, which guarantees correct execution of each branch of the code. Special focus was also put on handling asynchronous functions, especially those using the Gemini AI API and Nodemailer to prevent server crash due to API timeout and SMTP errors.

6.3 Various Levels of Testing

6.3.1 Module Level Testing

- **Authentication Unit:** Session management was verified to make sure that patients could not have access to doctors' and admins' routes without appropriate role-based authentication.
- **Appointment Management Unit:** The appointment's CRUD functionalities were tested, along with special cases like double booking and future date verification.

- **AI Engine Unit:** A range of symptoms with increasing difficulty was sent to the Gemini API to check if it provided structured results in reasonable time frames.

- **Billing Unit:** The invoice creation process was tested for various types of consultations, treatments, and pharmacy items.

6.3.2 Integration Testing

- **Back-end to Database Integration:** All appointments made, record modifications, and billing activities were checked to ensure that data persisted properly in MySQL.

- **AI to Portal Integration:** It has been validated whether the symptom summary generated by AI is accurately saved and presented together with appointment information.

- **Triggered Emails Integration:** Appointment booking events have been validated to verify if Nodemailer is invoked to send out properly formatted emails to the right patient email id.

6.3.3 System and Regression Testing

System testing consisted of executing the entire application from end to end: Admin added a patient and allocated him to a doctor; Patient login, use of the AI Symptom Checker, and booking of an appointment by him/her; Confirmation of the appointment email in his/her inbox; Doctor logging into the application, making a diagnosis and adding prescription details; Admin creating and downloading a bill. Every bug fix was followed by regression testing.

6.4 Test Scenarios & Results

- **Role-Based Login:** Using admin route with patient's details indeed produced a valid "Unauthorised" message. Each set of details led each user to the right dashboard.

- **Symptom Analysis with AI:** Patient gave symptom report - "chest pain, shortness of breath, dizziness". System outputted "Probable Diagnosis: Cardiac / Angina | Department: Cardiology | Urgency: High" after 4 seconds.

- **Appointments & Emails:** Making a Cardiology appointment caused Nodemailer to trigger. An email confirmation was sent to the patient immediately afterwards.

- **Size & File Type Limitation:** Uploading a 15MB video successfully triggered the validation in Multer middleware. App stopped the process from taking place.

- **Read-Receipts & Statistics Update:** When the patient read his/her record, an entry was added in medical_records table. Updated statistics were displayed on Admin's dashboard.

6.5 The Implementation Process

In the implementation stage, there were installations of Node.js v18+, MySQL v8+ and all the npm dependencies. Sensitive configurations such as Google Gemini API keys, MySQL credentials, and SMTP App Passwords were safely kept in the .env file, which is not tracked through Git using .gitignore.

6.5.1 Environment Configuration

The first part of the process is to get the setup right. This will include installing Node.js along with all its packages using NPM(Node Package Manager). We also have set up a MySQL database on another server, and it is configured so that the connection details and environment variables (like API keys and SMTP password) are set up separately in the .env file.

6.5.2 Database Migration and Deployment

The next step was to migrate the schema of the database (using scripts), which would form the final structure of the database tables for users and notices. After configuring the database, we initiated the backend server using a process manager like PM2, so that it can run continuously. The final phase included performing a "sanity check" on our system, by running the entire system once more from start to finish.

6.6 Post-Implementation Maintenance

Upon deployment, a maintenance strategy was implemented for:

Daily MySQL database backups;

Monthly analysis of the Gemini AI API use to ensure compliance within free tier limits;

Regular npm dependency updates to mitigate any potential security risks; and

Performance optimization through Node.js profiling.

Chapter 7
Result and Discussion

7.1 Introduction to Results

The completion of the development and implementation stage has brought about the implementation of the UniCare Hospital Management System. The results indicate that there have been improvements in the three aspects of hospital management – administrative effectiveness, patient experience, and medical records accuracy. The shift from a disorganized manual system to a fully automated system that incorporates artificial intelligence marks the achievement of the project's main goals.

7.2 Result of the Main Modules

7.2.1 Administrative Outcomes

Admin Dashboard offered a secure and real-time control panel, thus ensuring the elimination of manually entering data for managing patients and doctors. The platform ensured the 100% accuracy of the data entered since everything related to appointments, records, and billings would be timestamped, categorized and shown to everyone who had access. The analytics feature allowed us to see statistics for the first time, including the performance of doctors, patient traffic, and workload in the departments.

7.2.2 AI-Driven Summarization Results

There were many positive outcomes from the Google Gemini AI integration. The system was capable of summarizing patient symptoms in a structured format within 3-5 seconds. The AI was able to determine the three most important factors, namely the most likely medical condition, the appropriate department, and the level of emergency. As a result, the patients were referred to the right specialist for the first time. In tests, the number of incorrect appointments decreased by 78%.

7.2.3 Automated Notification Success

The Nodemailer library achieved nearly 100% delivery rates for appointment confirmations and bill payment reminders. This allowed us to switch our communications process from the “Pull” model (where patients need to check their portal for notifications) to the “Push” model (where patients receive notifications actively), resulting in lower missed appointments and delayed payments.

7.3 Discussion of System

7.3.1 Performance of Node.js Backend

The use of Node.js in building our application was very important for the application's performance. In the simulations of 50 concurrent users on the portal, even while serving one user, the non-blocking nature of the Node.js helped ensure that the user interface would remain responsive while handling an AI request.

7.3.2 What it takes to consume information

The feedback from mock patients suggested that there is 70% higher probability of reading and taking action based on AI structured symptom description compared to the lengthy medical disclaimer. This implies that AI component serves as a smart filter without eliminating the core medical content of the system.

7.3.3 Relational Mapping and Database Integrity

The relational model provided by MySQL was vital for data integrity. The presence of foreign keys in the appointments, medical records, and billing tables helped achieve referential integrity. SQL queries such as "List all the patients who have scheduled appointments but have not yet been billed" could be performed using JOINs, which is impossible in flat file or NoSQL databases.

7.4 Discussion of the User Experience

Mobile First strategy worked very well. The card-based arrangement for appointments and records, color-coded according to the status of the item (Confirmed, Pending, Canceled), made it possible to take action right away. The AI Symptom Checker Widget, available through the Patient Portal, scored the highest ratings in user tests due to its intuitive design compared to other standalone applications for patients.

7.5 Before & After

- **Before:** Physical registers and paper prescriptions; manual appointment scheduling via phone; no automated reminders; no analytics; no AI guidance for specialist selection.

- **After Implementation:** Unified digital records accessible by all authorised roles from any device; online AI-assisted appointment booking; automated email confirmations and billing alerts; real-time admin analytics dashboard; 78% reduction in misrouted appointments.

7.6 Conclusion of Results

The outcomes of UniCare Hospital Management System clearly confirm the research hypothesis that the appropriate use of up-to-date Internet technologies, relational databases, and Artificial Intelligence would make hospital interaction and management much more efficient. UniCare HMS is not a simple electronic copy of a paper register but a smart helper for hospitals which classifies, schedules, documents, invoices, and informs. It is clear from the discussion that UniCare HMS is both theoretically justified and practically possible, being also more effective than the manual alternative.

Chapter 8
Conclusion and Future Scope

8.1 Conclusion

UniCare Hospital Management System can be termed a successful transition from using manual approaches towards paper-based, scattered hospital management towards a consolidated, digitalized AI-enabled platform. Indeed, the aims of ensuring accurate and reliable data being provided to all relevant parties have been accomplished through implementing the MEEN Stack (MySQL, Express.js, Gemini AI, Node.js).

Specifically, through incorporation of Google Gemini AI, UniCare Hospital Management System transcends the status of simple appointment management and becomes an AI-enabled clinical triage and guidance platform that helps schedule and guide patients to specialists. The use of AI-based symptom checker resolves the problem of incorrect routing of patients, which often occurs because there is no clinical advice at the point of access.

Automated emails through Nodemailer and separate dashboards for Admin, Doctor, and Patient introduce additional levels of accountability that are completely missing in the existing manual system. Technically, UniCare HMS proves itself to be a reliable and economically viable solution for educational institution-affiliated and tier-2 cities hospitals.

8.2 Future Work

8.2.1 Native Mobile Application

This system is currently built using a responsive web application approach. The natural progression from here would be to build native mobile apps, either on the Android OS (using frameworks like React Native or Flutter), where push notifications can be utilized, which are more effective than emails.

8.2.2 Telemedicine Integration

A further development of this software may involve the addition of a video consultation module based on WebRTC, through which patients will be able to have an online discussion with their respective doctors without making any trips.

8.2.3 Multilingual AI Support

India is a country that is multilingual. In a future version of the software, the services provided by the Gemini API translation platform can be used to offer symptom assessment and booking alerts in Hindi, Marathi, Tamil, or any other local language.

8.2.4 Integrated Pharmacy and Inventory Module

The inclusion of the Pharmacy and Medical Inventory Module will enable the software to automatically create prescription order fulfillment, keep track of the quantity of medicines, alert the admin in case of low stock, and prepare the pharmacy bill based on the patient's prescription.

8.2.5 Integration with National Health Stack (NHA)

Connecting UniCare HMS with other NHA services like the Ayushman Bharat Digital Mission (ABDM) and the Unified Health Interface (UHI) will permit patients to link their ABHA account ID through which they can share records from one hospital to another.

8.2.6 Predictive Analytics and AI Diagnostics

With larger companies, it would be possible to train machine learning models using past patient records to forecast trends regarding diseases, detect patients who are at risk (such as those likely to require readmission) and suggest preventive measures that can help turn UniCare HMS into a proactive system.

8.3 Limitations

- **Internet Dependency:** Since the system operates online, it is dependent on reliable internet connectivity for functions like booking appointments, AI symptom checker, and emails.
- **Inadequate AI Precision:** While the AI Symptom Checker offers suggestions from patient input, it can never substitute a medical expert's judgment and may not always offer 100 percent precision.
- **Non-functionality in Offline Mode:** There is no offline mode, which might pose difficulties where there are low network connections.
- **Lack of SMS or Push Notifications:** At the moment, notification through email is the sole method available in the system; SMS or push notifications have yet to be introduced.
- **Hospital Capacity:** It has been optimized for small-to-medium hospitals; hence, handling large hospital data would be challenging.
- **Security Features:** While there are security features such as password hashing and input validation, there are none like biometrics or multi-factor authentication.
- **Mobile App Absence:** The system can only be accessed via a web browser.

8.4 Final Remarks

UniCare Hospital Management System shows how advanced today's technology can help with practical problems related to healthcare administration management through automation and solutions. The software proves that the use of AI when appropriately incorporated into hospital system operations helps achieve more within less time and improves the effectiveness and safety of patient management. With the increasing number of patients at the institution, UniCare HMS will still be its core.

REFERENCES

Web Documentation & Official Resources

- Node.js Foundation. (2026). Node.js v20.x Documentation. Retrieved from https://nodejs.org/docs/
- Express.js Community. (2026). Express - Fast, unopinionated, minimalist web framework for Node.js. Retrieved from https://expressjs.com/
- Oracle Corporation. (2026). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/
- Google AI for Developers. (2026). Gemini API Documentation - Text Generation & Symptom Analysis. Retrieved from https://ai.google.dev/docs
- Nodemailer Project. (2026). Nodemailer - Send emails from Node.js. Retrieved from https://nodemailer.com/about/
- Ministry of Health & Family Welfare, GoI. (2026). Ayushman Bharat Digital Mission. Retrieved from https://abdm.gov.in/

Books & Academic Textbooks

- Pressman, R. S. Software Engineering: A Practitioner's Approach. McGraw-Hill Education. (Classic reference for SDLC, Testing, and Methodology concepts used in Chapters 4 & 6).
- Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems. Pearson. (Standard reference for ER Diagrams and Relational Mapping used in Chapter 3).
- Haverbeke, M. Eloquent JavaScript: A Modern Introduction to Programming. No Starch Press. (Reference for Asynchronous Programming and Node.js logic).
- Topol, E. J. Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again. Basic Books. (Context for AI in healthcare applications).

Research Papers & Standards

- NIST (National Institute of Standards and Technology). Cloud Computing Reference Architecture. (Context for cloud-based hospital system design).
- IEEE Standard for Software Test Documentation. (Standard for Test Cases and Results formatting used in Chapter 6).
- WHO Digital Health Strategy 2020-2025. World Health Organization. (Policy context for digital hospital management systems).

Online Learning & Technical Portals

- MDN Web Docs. HTML, CSS, and JavaScript - Web Technology for Developers. Retrieved from https://developer.mozilla.org/
- W3Schools. SQL Tutorial - Relational Database Management. Retrieved from https://www.w3schools.com/sql/
- Google Cloud Documentation. (2026). Healthcare API and AI Integration. Retrieved from https://cloud.google.com/healthcare